

Hospital Management System Project Report

1. Student Details

Name: Saboo Riddesh Rajan

Roll Number: 24f1001102

Email: 24f1001102@ds.study.iitm.ac.in

About Me: I am a Diploma level student of the IIT Madras BS Degree program with an interest in full-stack web application development and Mobile application development . I enjoy building applications that combine backend logic, database design, and clean user interfaces

2. Project Details

Project Title: Hospital Management System

Problem Statement:

To build a Hospital Management System (HMS) web application that allows Admins, Doctors, and Patients to interact with the system based on their roles.

Approach:

The application is developed using Flask as the backend framework with SQLAlchemy ORM for database interaction. The system supports three primary user roles: Admin, Doctor, and Patient, each having dedicated dashboards and access control.

3. AI/LLM Declaration

I used ChatGPT for limited assistance such as understanding Flask-SQLAlchemy relationships, improving code structure, resolving formatting issues, and documentation formatting. The overall AI/LLM usage in this project is estimated to be around 10–15%.

All core implementation logic, integration, debugging, and feature development were completed manually.

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework

Flask-SQLAlchemy	ORM for SQLite database
SQLAlchemy	Database abstraction layer
Jinja2	Template engine for dynamic HTML rendering
Bootstrap 5	Frontend styling and responsive design
SQLite	Lightweight local database
bcrypt	Secure password hashing

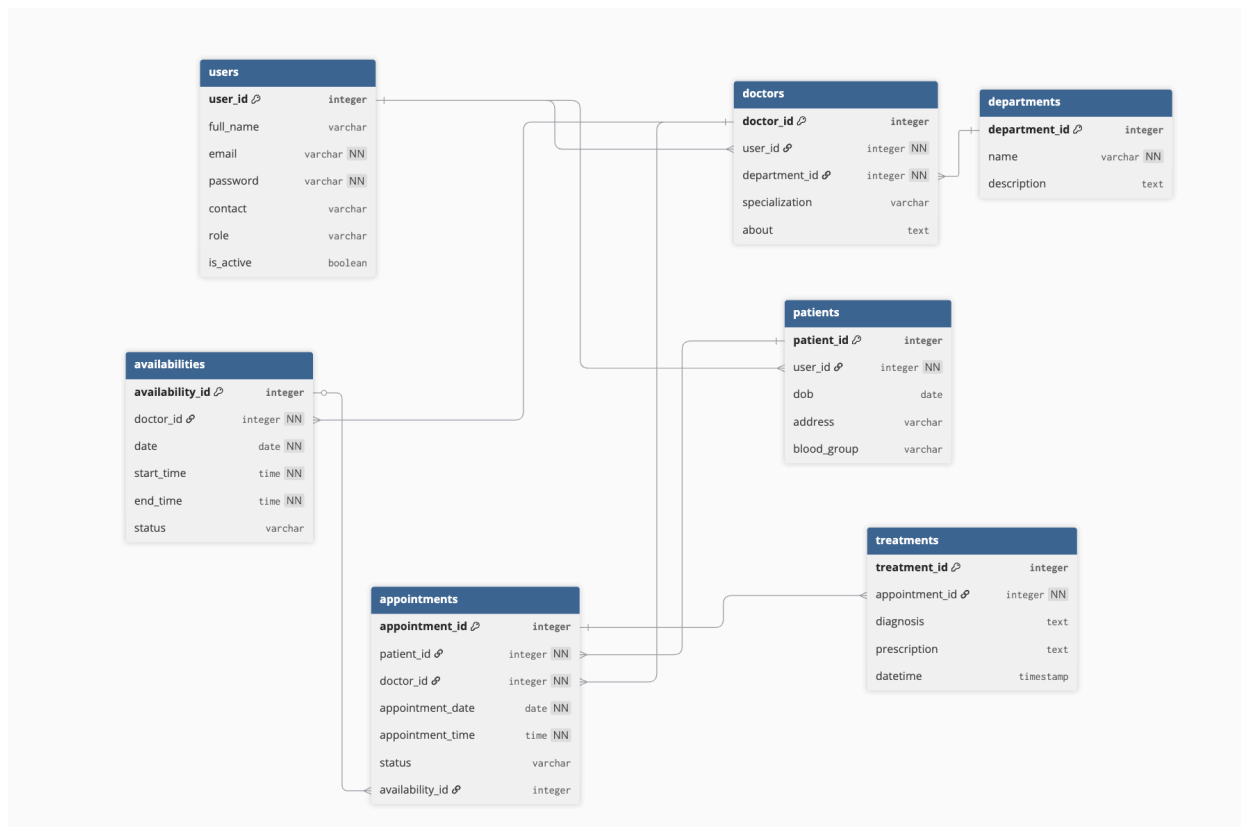
5. Database Schema / ER Diagram

Tables:

1. **User** — stores user profile and authentication details
2. **Department** — stores information about the departments in the hospital
3. **Doctor** - stores doctor details
4. **Patient** - stores patient details
5. **Availability** - stores doctor availability schedule
6. **Appointment** - Stores the appointments and their statuses
7. **Treatment** - stores diagnosis and prescription details for each appointment .

Relationships:

- One-to-One → User → Doctor
- One-to-One → User → Patient
- One-to-Many → Department → Doctor
- One-to-Many → Doctor → Availability
- One-to-Many → Doctor → Appointment
- One-to-Many → Patient → Appointment
- One-to-One → Appointment → Treatment



6. API Resource Endpoints

Common Routes

- / – Home page of the application
- /register – Patient registration (GET, POST)
- /login – User login (GET, POST)
- /logout – User logout (GET)

Admin Module Routes

- /admin_dashboard – Admin dashboard
- /admin_dashboard/add_doctor – Add new doctor (GET, POST)
- /admin_dashboard/add_department – Add new department (GET, POST)

- /admin_dashboard/view_doctors – View all doctors
 - /admin_dashboard/view_departments – View all departments
 - /admin_dashboard/view_patients – View all patients
 - /admin_dashboard/view_appointments – View all appointments
 - /admin_dashboard/departmentwise_doctors/<int:department_id> – View doctors by department
 - /blacklist_user/<int:user_id> – Blacklist a user
 - /unblacklist_user/<int:user_id> – Unblacklist a user
 - /delete_user/<int:user_id> – Delete a user
 - /admin_dashboard/patient_history/ – View patient medical history
-

Patient Module Routes

- /patient_dashboard – Patient dashboard
 - /patient_dashboard/department_details – View available departments
 - /patient_dashboard/doctor_availability – View doctor availability
 - /patient_dashboard/book_appointment – Book appointment
 - /patient_dashboard/cancel_appointment/ – Cancel appointment
 - /patient_dashboard/appointment_details/ – View appointment details
 - /patient_dashboard/view_history – View treatment and appointment history
-

Doctor Module Routes

- /doctor_dashboard – Doctor dashboard
- /doctor_dashboard/provide_availability – Provide availability for next 7 days (GET, POST)
- /doctor_dashboard/save_availability – Save availability (POST)
- /doctor_dashboard/update_patient_history – Update diagnosis and prescription (POST)

- `/doctor_dashboard/mark_appointment_completed` – Mark appointment as completed (POST)
 - `/doctor_dashboard/cancel_appointment/` – Cancel appointment as doctor (POST)
 - `/doctor_dashboard/view_patient_history` – View patient medical history
-

7. Architecture and Features

Architecture Overview

The application follows a **modular Flask-based architecture** with proper separation of concerns between backend logic, database models, templates, and static files.

- **app.py** – Main Flask application entry point that contains all routes, request handling, and core logic.
- **models.py** – Contains all database models such as User, Doctor, Patient, Department, Appointment, Availability, and Treatment using SQLAlchemy .
- **extensions.py** – Initializes and configures the SQLAlchemy database instance.
- **/templates** – Contains all Jinja2 HTML templates used for Admin, Doctor, and Patient dashboards.
- **/static** – Stores static frontend assets (images)
- **/instance** – Stores instance-specific data such as the SQLite database file.
- **/venv** – Virtual environment used for dependency isolation (not included in deployment).
- **requirements.txt** – Lists all external Python dependencies.
- **README.md** – Project documentation and setup instructions.

Key Features Implemented

- Secure user authentication with role-based access control.
 - Separate dashboards for **Admin**, **Doctor**, and **Patient**.
 - Department and doctor management.
 - Doctor availability scheduling for the next 7 days.
 - Appointment booking and cancellation system.
 - Update and View Prescriptions.
 - Patient medical history tracking.
 - Appointment and prescription printing feature.
 - Auto cancellation of past appointments .
-

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1orgskBqV6o3GMDreot-sqD8KuuUgWKh7/view?usp=sharing>